PROJECT DOCUMENTATION

RAG-Powered Enterprise Document Assistant

Implementation: DeepSearch Cloud File Manager

TEAM PHOENIX | Team No. 93

S.No	Name	Registration No.
1	V. Gowtham Reddy	99230040796
2	V. Lakshmi Kanth Reddy	99230040233
3	K.E. Madhu Sudhan Goud	99230040330
4	A. Siva Ganesh Reddy	99230040262
5	K. Pavithra	99230040320

DeepSearch Cloud File Manager

AI-powered deep search over a bounded, private corpus of PDF, Word, Excel, CSV, text and image files. Users upload documents, the system extracts and indexes them, and then answers questions with hybrid retrieval (semantic + lexical) and guarded, citation-backed LLM answers.

Table of contents

1. Overview
2. Repository structure
3. System architecture
4. Tech stack
5. Core workflows
6. Backend reference
7. REST API reference
8. Database schema
9. Guardrails and security
10. Frontend reference
11. Configuration (environment variables)
12. Local development
13. Deployment on Render
14. Testing and CI
15. Operational notes and known caveats

1. Overview

What it does

- Registers and authenticates users with email OTP verification.
- Accepts uploads of `.pdf`, `.docx`, `.xlsx`, `.xls`, `.csv`, `.jpg`, `.jpeg`, `.png`, `.txt`, `.md`.
- Extracts text (with OCR for scanned PDFs and images), splits it into source-referenced chunks, embeds them, and indexes them.
- Provides ranked search results with evidence text and source location (page / sheet / row reference).
- Generates short natural-language answers via Groq, restricted to retrieved evidence, with validated citations.
- Offers per-file and corpus-wide chat, search history, and a retrieval-quality evaluation page.

Design principles

- **Private per-user corpus** — every query, index and file is scoped to the authenticated user.
- **Grounded answers only** — the LLM never searches; it only explains evidence the retrieval engine selected.
- **Persistence without a persistent disk** — originals live in object storage (or the database), embeddings live in the database, and FAISS indexes are rebuilt from stored embeddings on demand.

2. Repository structure

```
Gen-AI/
■■■ README.md                  # Short overview + quick start
■■■ render.yaml                # Render Blueprint (API, worker, Redis, static site, Postgres)
■■■ .github/workflows/
■   ■■■ deepsearch-ci.yml      # CI: backend compile + tests, frontend build
■■■ docs/
■   ■■■ ARCHITECTURE.md        # High-level workflow note
■■■ backend/
■   ■■■ requirements.txt
■   ■■■ render.yaml            # Backend-only Blueprint variant
■   ■■■ .env.example
■   ■■■ app/
■   ■   ■■■ main.py            # FastAPI app, all API routes
■   ■   ■■■ config.py          # Settings loaded from environment
■   ■   ■■■ db.py              # SQLite/PostgreSQL layer + schema
■   ■   ■■■ security.py        # scrypt password hashing, JWT helpers
■   ■   ■■■ otp.py             # OTP generation / hashing / limits
■   ■   ■■■ email_service.py   # Brevo (HTTPS) email sender, OTP template
■   ■   ■■■ guardrails.py      # Validation, injection + secret defenses
■   ■   ■■■ ingest.py          # Text extraction + OCR
■   ■   ■■■ chunking.py        # Chunking with source references
■   ■   ■■■ embeddings.py      # all-MiniLM-L6-v2 via ONNX Runtime
■   ■   ■■■ vector_index.py    # Per-user FAISS index (rebuilt from DB)
■   ■   ■■■ search.py          # Hybrid retrieval, scoring, Redis cache
■   ■   ■■■ ai_service.py      # Groq prompt + guarded answer generation
■   ■   ■■■ storage.py         # Object storage adapter (local / S3)
■   ■   ■■■ job_queue.py       # Redis-backed ingestion queue + recovery
■   ■   ■■■ ingestion_worker.py# Upload-job processing pipeline
■   ■   ■■■ worker.py          # Standalone worker entry point
■   ■   ■■■ worker_http.py     # HTTP wrapper (/health, /wake) for Render
■   ■   ■■■ evaluation.py      # Benchmark + before/after snapshots
■   ■■■ tests/
■       ■■■ test_chunking.py
■       ■■■ test_guardrails.py
■■■ frontend/
    ■■■ package.json
    ■■■ vite.config.ts
    ■■■ index.html
    ■■■ .env.example
    ■■■ scripts/prerender-routes.mjs  # Direct SPA entry points per route
    ■■■ src/
        ■■■ main.tsx           # All pages and routing
        ■■■ api.ts             # API client + token handling
        ■■■ styles.css
```

3. System architecture

```
User
 v
React / Vite frontend (static site)
 v  HTTPS + Bearer JWT
FastAPI backend (deepsearch-api)
 |-- PostgreSQL (SQLite locally)  -> users, files, chunks, embeddings, jobs, chats, history
 |-- Object storage (S3 / R2 / MinIO / local / DB blobs) -> original uploaded files
 |-- FAISS + all-MiniLM-L6-v2     -> semantic retrieval
 |-- BM25-style lexical + exact-match scoring -> names, numbers, exact terms
 |-- Groq LLM                     -> guarded, evidence-grounded explanations
 `-- Redis / Render Key Value     -> ingestion queue + short-lived search cache
       ^
Ingestion worker (deepsearch-ingestion-worker)
 `-- Extract/OCR -> chunk -> embed -> store -> index
```

Two services, one codebase. Long-running OCR and embedding work is isolated from the API in a separate worker service so uploads do not block or crash API requests. The API enqueues a job and "wakes" the worker (needed on Render's free tier, which sleeps idle web services).

4. Tech stack

Layer	Technology
Frontend	React, TypeScript, Vite, Framer Motion, three.js / react-three-fiber / drei, lucide-react
API	FastAPI, Uvicorn, Pydantic v2
Auth	scrypt password hashing, HS256 JWT (PyJWT), email OTP
Database	PostgreSQL (`psycopg2-binary`) in production, SQLite for local dev
Storage	boto3 S3-compatible adapter, local filesystem, or database blobs
Extraction	PyMuPDF (PDF), python-docx, pandas + openpyxl (XLSX/XLS/CSV), RapidOCR (scanned PDF pages, images)
Embeddings	Xenova/all-MiniLM-L6-v2 (ONNX Runtime + `tokenizers`)
Vector search	faiss-cpu (`IndexFlatIP`, per-user)
LLM	Groq (`groq` SDK), default model `qwen/qwen3.8-27b`
Queue / cache	Redis (`redis-py`)
Email	Brevo transactional email API (HTTPS)
CI / Hosting	GitHub Actions, Render

5. Core workflows

5.1 Registration and login

1. `POST /api/auth/register` creates an unverified user and emails a 6-digit OTP.
2. `POST /api/auth/verify-registration` validates the OTP, marks the user verified, and returns an access token.
3. `POST /api/auth/login` returns an access token (only for verified users).
4. Password reset: `forgot-password` → `verify-reset-otp` → `reset-password`.

OTP rules: 6 digits, stored only as a SHA-256 hash, valid for 10 minutes, 5 attempts max, resend cooldown of 60 seconds.

5.2 Upload and indexing

```
Upload (POST /api/files/upload)
 v validate extension / size / non-empty
 v SHA-256 content hash -> per-user duplicate check (409 on duplicate)
 v ZIP-container inspection (DOCX/XLSX "archive-bomb" check)
 v persist original to object storage
 v create file row + upload_job row, enqueue job, wake worker
Worker:
 extracting (25%) -> chunking (45%) -> indexing (60%) -> ready (100%)  (or failed)
```

The frontend polls `GET /api/files/upload-jobs/{job_id}` for `status`, `stage`, `progress`, `error`.

- **PDF extraction:** native text first; OCR is only used as a fallback for image-only pages.
- **Images:** OCR directly.
- **Chunking:** windows of up to ~900 characters with ~120 characters of overlap; numbered items/paragraphs are kept together where possible, and tabular "problem rows" (number, name, difficulty, details) become atomic chunks with structured metadata.
- Each chunk stores `content`, `source_ref` (e.g. page/sheet reference), `metadata` (JSON) and `embedding` (JSON vector).
- After indexing, the user's search cache version is bumped and the FAISS index is rebuilt lazily from stored embeddings.

5.3 Search

`search(user_id, query, limit, file_ids=None)` in `search.py`:

1. Check the Redis cache (60-second TTL, keyed by user, cache version, limit, scope and normalized query).
2. **Semantic:** embed the query, search the user's FAISS index (inner product on normalized vectors), normalize cosine score to 0–1.
3. **Lexical:** BM25-style term scoring over chunk text, min-max normalized.
4. **Exact relevance:** bonus for exact phrase match and term coverage; query features such as "top N" and difficulty words (easy / medium / hard) are matched against chunk metadata.
5. **Fusion:** `score = 0.50·semantic + 0.25·lexical + 0.25·exact` plus a small bonus for metadata matches; scores are then normalized to a 0–0.99 relevance value.

6 Each result carries file info, `source_ref`, evidence text, and component scores. Engine label: `hybrid-semantic-lexical-reranked`.

5.4 Guarded answers ("Deep search" and chat)

1 Query is validated (max 1000 chars) and checked for prompt-injection patterns.

2 Top evidence chunks are retrieved (up to 12 supplied to the LLM) and sanitized (secret redaction, length limits — see `safe_evidence_context`).

3 Groq is prompted with a strict contract: answer only from evidence, treat evidence as untrusted data, respect "top N" and difficulty metadata, keep summaries to 1–3 sentences, and return JSON `{answer, confidence, citations[]}`.

4 The response shape and every citation are validated against the retrieved chunks. On failure (or no `GROQ_API_KEY`) the system returns a grounded fallback built directly from evidence.

5 Deep-search answers are saved to `search_history`.

6. Backend reference

Module	Responsibility
`main.py`	App creation (FastAPI, CORS), request models, rate limiting, auth dependency, all routes, worker wake logic
`config.py`	Frozen Settings dataclass populated from environment variables
`db.py`	Connection handling for SQLite/PostgreSQL (`?` placeholders auto-converted), schema creation (`init_db`), helpers `one`, `all_`, `exe`, `jd`, `jl`, `now`
`security.py`	`hash_password` / `verify_password` (scrypt, n=2^14, r=8, p=1), `create_token` / `decode` (HS256 JWT with type and jti)
`otp.py`	OTP constants, generation (`secrets`), hashing
`email_service.py`	Sends OTP emails through Brevo's HTTPS API
`guardrails.py`	`validate_query`, `guard_ai_query`, `detect_prompt_injection`, `redact_sensitive`, `validate_upload`, `inspect_zip_container`, `safe_evidence_context`, `validate_ai_citations`, `grounded_fallback`
`ingest.py`	`extract(path)` → (text, refs, ocr_used, pages); lazy-loaded RapidOCR engine with explicit release to save memory
`chunking.py`	Source-aware chunk generation
`embeddings.py`	Lazy ONNX model download/cache (under `HF_HOME`), `embed_query`, status reporting
`vector_index.py`	In-memory per-user FAISS `IndexFlatIP`; `rebuild_user_index`, `semantic_search`
`search.py`	Hybrid retrieval, scoring, Redis cache + version bumping
`ai_service.py`	Groq client, system prompt, JSON parsing, `answer_with_guardrails`
`storage.py`	`ObjectStorage` with `upload_file`, `download_file`, `delete`; backends: local, s3, database
`job_queue.py`	Versioned Redis queue (`deepsearch:ingestion:jobs:v4`), pending set, stale-job recovery (60 s heartbeat threshold), worker thread
`ingestion_worker.py`	`_process_upload_job` — extract → chunk → embed → index pipeline with progress updates and heartbeats
`worker.py` / `worker_http.py`	Worker entry point and its small FastAPI app (`/`, `/health`, `/wake`)
`evaluation.py`	Builds benchmark cases from the user's corpus, compares hybrid search against a keyword baseline, computes metrics, and snapshots/compares corpus state

7. REST API reference

All routes except health and auth require `Authorization: Bearer <access_token>`. Errors are returned as `{"detail": "..."}`.

Health

Method	Path	Description
GET	`/`	Basic status
GET	`/api/health`	Version, DB type, storage backend, embedding status, guardrail flags, email/worker configuration

Authentication

Method	Path	Body	Rate limit / min
POST	`/api/auth/register`	full_name (2–100), email, password (8–128)	10
POST	`/api/auth/verify-registration`	email + OTP	—
POST	`/api/auth/resend-registration-otp`	email	6
POST	`/api/auth/login`	email, password	8
POST	`/api/auth/forgot-password`	email	6
POST	`/api/auth/verify-reset-otp`	email + OTP	—
POST	`/api/auth/reset-password`	reset token + new password	—
GET	`/api/auth/me`	—	—

Files

Method	Path	Description
POST	`/api/files/upload`	Multipart upload (`file`). Max 25 MB. Returns file + job info. 409 if identical content already uploaded. Limit 20/min
GET	`/api/files/upload-jobs/{job_id}`	Ingestion job status (status, stage, progress, error)
GET	`/api/files`	List the user's files
GET	`/api/files/{file_id}`	File details
GET	`/api/files/{file_id}/chunks?limit=50`	Inspect extracted chunks
POST	`/api/files/{file_id}/reindex`	Re-extract and re-index a file
DELETE	`/api/files/{file_id}`	Delete file, chunks, and stored original
GET	`/api/corpus/stats`	Corpus statistics

Search and AI

Method	Path	Body	Rate limit / min
POST	`/api/search`	query (≤1000), limit (1–50)	60
POST	`/api/deep-search`	query, limit (1–20) → results + guarded answer, saved to history	20
POST	`/api/ai/answer`	query, limit (1–12)	20

Chat and history

Method	Path	Description
GET	`/api/chats?file_id=`	List chat threads/messages (corpus-wide, or scoped to one file)
POST	`/api/chats/message`	message (≤2000), optional thread_id, file_id. Limit 20/min
GET	`/api/history?limit=50`	Past deep-search queries/answers
DELETE	`/api/history/{history_id}`	Remove a history entry

Evaluation

Method	Path	Description
GET	`/api/evaluation/benchmark?limit=6`	Run a retrieval benchmark on the user's corpus
POST	`/api/evaluation/snapshot`	Capture top results for a query
POST	`/api/evaluation/compare`	Compare a prior snapshot with the current state (shows how results adapt after new uploads)

8. Database schema

Created automatically by `init_db()` on API startup. SQLite uses TEXT / INTEGER; PostgreSQL is used in production (`file_blobs.content` is BYTEA).

Table	Purpose	Key columns
`users`	Accounts	id, email (unique), full_name, password_hash, is_verified, created_at
`refresh_tokens`	Refresh-token records	jti, user_id, token_hash, expires_at, revoked_at
`otp_challenges`	Email OTPs and reset tokens	user_id, email, purpose, otp_hash, expires_at, attempts, used, reset_jti, reset_expires_at, reset_used
`files`	File metadata	id, user_id, name, mime_type, path (storage URI/key), status, ocr_used, page_count, chunk_count, content_hash (unique per user + type)
`file_blobs`	Original bytes when STORAGE_BACKEND=database	object_key, content, content_type
`chunks`	Searchable text units	id, file_id, user_id, content, source_ref, metadata (JSON), embedding (JSON vector)
`upload_jobs`	Ingestion progress	id, user_id, file_id, name, status, stage, progress, error, result
`search_history`	Saved deep-search results	query, answer, confidence, citations, result_count
`chat_threads`	Chat conversations	user_id, scope_type, file_id, title
`chat_messages`	Messages	thread_id, role, content, citations, confidence

9. Guardrails and security

Input limits (guardrails.py)

Guardrail	Value
Max query length	1,000 characters
Max upload size	25 MB
Allowed extensions	.pdf .docx .xlsx .xls .csv .jpg .jpeg .png .txt .md
Max uncompressed ZIP-container size (DOCX/XLSX)	150 MB
Max ZIP members	2,500
Max LLM context / answer	12,000 / 5,000 characters

LLM safety

- Prompt-injection detection on user queries (e.g. "ignore previous instructions", "reveal the system prompt", "you are now a…").
- Secret redaction applied to evidence and answers: private keys, `sk-…` API keys, GitHub tokens, AWS access keys, JWTs, and `password=` / `secret=` style strings.
- Evidence treated as data, never instructions.
- Citation validation: answers must cite chunks that were actually supplied; otherwise a grounded fallback is returned.

Auth and abuse protection

- Passwords hashed with scrypt and a random 16-byte salt; constant-time comparison.
- JWTs signed HS256, with token type and `jti`; access tokens last 15 minutes by default.
- Per-IP, in-memory rate limits per endpoint group (see API tables).
- Email verification required before login.
- Exact-content duplicate detection per user.
- All data queries are scoped by `user_id`.
- CORS allows the configured frontend origin, local dev origins, and `*.onrender.com` HTTPS hosts.
- Worker `/wake` endpoint is protected by `X-Worker-Wake-Token` when configured.

10. Frontend reference

Single-page React app (`frontend/src/main.tsx`) with client-side routing via the History API. `scripts/prerender-routes.mjs` copies `dist/index.html` into a folder per route after build so direct links work on static hosting; Render also rewrites `/*` → `/index.html`.

Route	Page	Notes
`/`	Home / landing	3D visuals, how-it-works sections
`/login`, `/register`	Auth	Redirect to /dashboard if already signed in
`/check-email`, `/verify-email`	Registration OTP flow	
`/forgot-password`, `/verify-reset`, `/reset-password`	Password reset flow	
`/dashboard`	Main workspace	Search, deep search, results with evidence
`/ingest`	Upload flow	Upload with live job progress
`/workspace`	File workspace	Browse files, inspect chunks, reindex, delete
`/chat`	Chat	Corpus-wide or per-file chat
`/history`	Search history	
`/evaluation`	Retrieval evaluation	Benchmark and before/after snapshots
`/about`	About	

Protected routes (`/dashboard`, `/evaluation`, `/history`, `/workspace`, `/ingest`, `/chat`) redirect to `/login` when no token is present.

API client (`api.ts`): wraps `fetch`, attaches `Authorization: Bearer <token>`, stores the token in `sessionStorage` under `deep_token`, and surfaces backend `detail` messages as errors.

11. Configuration (environment variables)

Backend

Variable	Default	Purpose
`FRONTEND_ORIGIN`	http://localhost:5173	Allowed CORS origin
`JWT_SECRET`	CHANGE-ME	Must be set to a strong random value in production
`ACCESS_TOKEN_MINUTES`	15	Access token lifetime
`REFRESH_TOKEN_DAYS`	7	Refresh token lifetime
`DATABASE_URL`	sqlite:///./deepsearch.db	Use a PostgreSQL URL in production
`UPLOAD_DIR`	system temp /deepsearch	Temporary ingestion workspace
`LOCAL_STORAGE_DIR`	backend/storage	Used when STORAGE_BACKEND=local
`STORAGE_BACKEND`	local	local, s3, or database
`S3_ENDPOINT_URL`, `S3_REGION`, `S3_BUCKET`, `S3_ACCESS_KEY_ID`, `S3_SECRET_ACCESS_KEY`	— / auto	S3-compatible storage (AWS S3, Cloudflare R2, MinIO)
`GROQ_API_KEY`	—	Enables LLM answers (fallback answers used if empty)
`GROQ_MODEL`	qwen/qwen3.8-27b	Groq model name
`EMAIL_PROVIDER`	brevo	Email transport
`BREVO_API_URL`		Brevo SMTP API URL
`BREVO_API_KEY`, `BREVO_SENDER_EMAIL`, `BREVO_SENDER_NAME`	— / — / DeepSearch	Sender credentials
`EMAIL_HTTP_TIMEOUT_SECONDS`	10	Email API timeout
`EMBEDDING_MODEL_REPO`	Xenova/all-MiniLM-L6-v2	Embedding model
`HF_HOME`	/tmp/.cache/huggingface	Model cache location
`REDIS_URL`	—	Ingestion queue and search cache
`INGESTION_WORKER_EXTERNAL_URL`	—	Worker URL the API wakes
`INGESTION_WORKER_WAKE_TOKEN`	—	Shared secret for /wake
`DEBUG`	true	Debug flag
`OMP_NUM_THREADS`, `ORT_INTRA_OP_NUM_THREADS`, `ORT_INTER_OP_NUM_THREADS`, `MALLOC_ARENA_MAX`	set to 1 / 2 in render.yaml	Keep memory low on small instances

Frontend

Variable	Purpose
`VITE_API_BASE_URL`	Backend URL (see caveats: api.ts currently hard-codes the production URL)

Never commit secrets. `.env` is already in `.gitignore`.

12. Local development

Prerequisites: Python 3.11, Node.js 20+.

Backend

```
cd backend
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env        # then edit values
uvicorn app.main:app --reload --port 8000
```

- The embedding model is downloaded from Hugging Face on first use.
- Without `BREVO_API_KEY` / `BREVO_SENDER_EMAIL`, OTP emails cannot be delivered (check `/api/health` → `email_configured`).
- Without `GROQ_API_KEY`, answers fall back to evidence-based responses.
- Ingestion is queue-driven (Redis). Set `REDIS_URL` and run a worker (see below) to process uploads.

Ingestion worker (local)

```
cd backend
python -m app.worker                    # background worker process
# or the HTTP-wrapped version used on Render:
uvicorn app.worker_http:app --port 8001
```

Frontend

```
cd frontend
npm install
npm run dev                             # http://localhost:5173
```

Set `VITE_API_BASE_URL=http://localhost:8000` in `frontend/.env`. Note that `src/api.ts` currently uses a fixed production URL; change the API constant to read `import.meta.env.VITE_API_BASE_URL` when developing locally.

Quick API check

```
curl http://localhost:8000/api/health
```

13. Deployment on Render

The root `render.yaml` Blueprint defines:

Resource	Type	Purpose
`deepsearch-api`	Web service (Python 3.11, Singapore)	FastAPI app — `uvicorn app.main:app`, health check `/api/health`
`deepsearch-ingestion-worker`	Web service	`uvicorn app.worker_http:app`, health check `/health`; processes upload jobs
`deepsearch-redis`	Key Value	Ingestion queue + search cache (LRU, no persistence)
`deepsearch-db`	PostgreSQL	Application data
`hackthon-for`	Static site	React build (`npm ci && npm run build`, publish `dist`, SPA rewrite)

Steps

1. Push the repo to GitHub and create a Render Blueprint from it.
2. Enter secrets marked `sync: false`: `GROQ_API_KEY`, `BREVO_API_KEY`, `BREVO_SENDER_EMAIL` (and S3 credentials if using S3).
3. `JWT_SECRET` and `INGESTION_WORKER_WAKE_TOKEN` are generated automatically.
4. Confirm `FRONTEND_ORIGIN` matches the deployed static-site URL.
5. The Blueprint uses `STORAGE_BACKEND=database` (originals stored in PostgreSQL) so it works without object storage. Switch to `s3` for larger or long-term deployments.

The free tier sleeps idle services; the API calls the worker's `/wake` endpoint and re-wakes it while jobs are stuck in the queue.

14. Testing and CI

Unit tests

Located in `backend/tests`, run with `pytest tests -q`.

- `test_chunking.py` — problem tables become atomic chunks.
- `test_guardrails.py` — query length, prompt-injection detection, secret redaction, grounded fallback requires evidence, upload extension validation, ZIP size guard.

GitHub Actions

`.github/workflows/deepsearch-ci.yml` on push/PR to `main`:

- **backend:** Python 3.11 → install requirements → `python -m compileall app` → `pytest tests -q`
- **frontend:** Node 20 → `npm install` → `npm run build`

15. Operational notes and known caveats

These are observations from reading the code, useful when maintaining or extending the project.

1. **Hard-coded API URL in the frontend.** `frontend/src/api.ts` fixes API to the production backend, so `VITE_API_BASE_URL` is currently ignored. Use the env variable for local development.
2. **JWT_SECRET default is `CHANGE-ME`.** Always override it outside local development.
3. **Rate limiting is in-memory per process.** Limits reset on restart and are not shared across instances; use Redis-based limiting if you scale out.
4. **Login issues an access token only.** The `refresh_tokens` table and `REFRESH_TOKEN_DAYS` setting exist, but no refresh endpoint is exposed; sessions end when the 15-minute access token expires.
5. **Token storage.** Tokens are kept in `sessionStorage`, so they clear when the browser tab closes.
6. **README vs. Blueprint storage mode.** The README highlights S3 storage, while `render.yaml` defaults to `STORAGE_BACKEND=database`.
7. **"Reranker" wording.** `docs/ARCHITECTURE.md` mentions a reranker; in code, final ranking is a weighted fusion of semantic, lexical and exact-match scores rather than a separate cross-encoder model.
8. **Memory sensitivity.** OCR and ONNX embeddings are memory-heavy; the single-thread environment settings and the separate worker service exist for this reason. Free-tier restarts can interrupt jobs; `job_queue.recover_jobs()` re-enqueues stale ones.
9. **Embeddings stored as JSON text.** Fine for a bounded corpus; for much larger corpora consider pgvector or a dedicated vector store.
10. **Frontend dependencies use `latest`.** Pinning versions (and committing a lockfile) would make builds reproducible; the Render build uses `npm ci`, which requires a lockfile.

Generated from the repository contents (backend, frontend, Blueprint and CI configuration).